Artificial and Computational Intelligence
Assignment 2

Problem statement 1 : Gaming

The game is played on a 7x6 grid show below. Players take turns dropping one of their colored discs from the top into a column. The disc can fall only into the lowest available space within the column. The first player to form an unbroken chain of four discs horizontally, vertically, or diagonally wins the game. The game is considered a draw if the board is filled without any player achieving a connect four. Your task include

1. Implement the basic game logic, including initializing the board, handling player moves, checking for a win or draw condition, and alternating turns between players
2. Implement the minimax algorithm to explore possible future game states. The system should be capable of understanding positive positions which can help in winning the game.

Problem statement 2: Logic

The Pragyan Rover from Chandrayan 3 must make decisions for measuring certain parameters from the moon (c0 or c1) and data comes from 10 sensors (Dataset attached). Use the below decision tree and create Prolog rules to predict which parameter to measure for the given condition. Take the attribute values from the user by giving suitable user prompts and predict the class

```
a5 = false: c0 (44.0/2.0)
a5 = true
| a8 = false
| |  a9 = false
| |  |  a2 = false: c0 (4.0/1.0)
| |  |  a2 = true
| |  |  |  a0 = false
| |  |  |  |  a4 = false: c1 (2.0)
| |  |  |  |  a4 = true: c0 (2.0)
|  |  |  |  a0 = true: c1 (5.0)
|  |  a9 = true: c1 (15.0/1.0)
| a8 = true
| |  a1 = false
| |  |  a2 = false
| |  |  |  a0 = false: c1 (4.0/1.0)
| |  |  |  a0 = true: c0 (3.0)
| |  |  a2 = true
| |  |  |  a4 = false: c1 (5.0)
|  |  |  |  a4 = true: c0 (2.0)
|  |  a1 = true: c0 (14.0/2.0)
```

Dataset : https://drive.google.com/file/d/1cFnSp-VDtqON7dQZPVgTTGZEPFljk2lk/view?usp=sharing

Evaluations will be based on the following.

1. Use Min-Max algorithm and implement the game in PYTHON (35% marks)
2. Derive the rules from the given decision tree and code as Prolog rules. (35% marks)
3. Interactive implementation. Dynamic inputs-based run of the game with step wise board display and error free game ending. (15% marks)
4. Interactive implementation. Dynamic inputs-based run of the logic expert system with step wise options display and error free recommendation & ending. (15% marks)

Important Note:

- You are provided with the python notebook template which stipulates the structure of code and documentation. Use well intended python code.
- Use a separate MS word document for explaining the theory part. Do not include the theory part in the Python notebook except Python comments.
- The implementation code must be completely original and executable.
- Please keep your work (code, documentation) confidential. If your code is found to be plagiarized, you will be penalized severely. Parties involved in the copy will be considered equal partners and will be penalized severely.